Nama: Arjun Zain Regeng Pangestu

NIM: 254107020144

Kelas: 1C

JOBSHEET 4

BRUTE FORCE DAN DIVIDE CONQUER

4.2.3. Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!
2. Apakah memungkinkan perulangan pada method `faktorialBF()` diubah selain menggunakan for? Buktikan!
3. Jelaskan perbedaan antara `fakto *= i;` dan int `fakto = n * faktorialDC(n-1);` !
4. Buat Kesimpulan tentang perbedaan cara kerja method `faktorialBF()` dan `faktorialDC()`!

1. Pada algoritma Divide and Conquer, bagian **if** digunakan sebagai kondisi dasar (*base case*), yaitu kondisi untuk menghentikan proses rekursi. Sementara itu, bagian **else** digunakan untuk menjalankan proses rekursif dengan cara memecah permasalahan menjadi bagian yang lebih kecil.
2. Ya, perulangan dapat menggunakan struktur lain, seperti while. Hal ini menunjukkan bahwa perhitungan faktorial tidak bergantung pada satu jenis perulangan tertentu.
3. Pernyataan fakto *= i; digunakan dalam proses iteratif (perulangan) untuk mengalikan nilai secara bertahap.
   Sedangkan int fakto = n * faktorialDC(n-1); digunakan dalam rekursi, yaitu dengan memanggil fungsi itu sendiri untuk menyelesaikan masalah
4. Metode Brute Force menggunakan pendekatan iteratif dengan perulangan sehingga lebih sederhana dan mudah dipahami. Sementara itu, metode Divide and Conquer menggunakan pendekatan rekursif dengan cara membagi masalah menjadi bagian yang lebih kecil sebelum digabungkan Kembali.

4.3.3. Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu `pangkatBF()` dan `pangkatDC()`!
2. Apakah tahap *combine* sudah termasuk dalam kode tersebut?Tunjukkan!
3. Pada method `pangkatBF()` terdapat parameter untuk melewatkan nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class `Pangkat` telah ada atribut `nilai` dan `pangkat`, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method `pangkatBF()` yang tanpa parameter?
4. Tarik tentang cara kerja method `pangkatBF()` dan `pangkatDC()`!

1. Method pangkatBF() menggunakan perulangan untuk menghitung hasil perpangkatan secara langsung. Sebaliknya, method pangkatDC() menggunakan pendekatan rekursif dengan prinsip Divide and Conquer.
2. Tahap *combine* sudah terdapat dalam kode tersebut, yaitu pada bagian yang menggabungkan hasil dari proses rekursi, misalnya dengan mengalikan hasil yang telah diperoleh sebelumnya.
3. Penggunaan parameter masih relevan karena memudahkan pengiriman nilai ke dalam method. Namun, parameter tidak selalu diperlukan apabila nilai yang digunakan sudah disimpan sebagai atribut dalam class. Dengan demikian, method tetap dapat dibuat tanpa parameter.
4. Method Brute Force bekerja dengan cara menghitung secara langsung menggunakan perulangan. Sementara itu, metode Divide and Conquer menyelesaikan masalah dengan cara membaginya menjadi bagian yang lebih kecil, kemudian hasilnya digabungkan kembali.

4.4.4. Pertanyaan

1. Kenapa dibutuhkan variable `mid` pada method `TotalDC()`?
2. Untuk apakah statement di bawah ini dilakukan dalam `TotalDC()`?

```
double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);
```

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```

4. Apakah base case dari `totalDC()`?
5. Tarik Kesimpulan tentang cara kerja `totalDC()`

1. Variabel mid digunakan untuk menentukan titik tengah array sehingga data dapat dibagi menjadi dua bagian, yaitu bagian kiri dan kanan, dalam proses Divide and Conquer.
2. Pernyataan tersebut berfungsi untuk memanggil proses rekursif pada bagian kiri dan kanan array, sehingga permasalahan dapat dipecah menjadi bagian yang lebih kecil.
3. Penjumlahan antara lsum dan rsum dilakukan untuk menggabungkan hasil perhitungan dari bagian kiri dan kanan array, sehingga diperoleh total keseluruhan.
4. Kondisi dasar (*base case*) terjadi ketika array hanya memiliki satu elemen. Pada kondisi ini, nilai elemen tersebut langsung dikembalikan tanpa proses rekursi lebih lanjut.
5. Metode ini bekerja dengan membagi array menjadi dua bagian (*divide*), kemudian menghitung masing-masing bagian (*conquer*), dan akhirnya menggabungkan hasilnya (*combine*). Pendekatan ini efektif untuk menangani data dalam jumlah besar.